

Dispositivos Conectados Protocolos IP para IoT: MQTT

2025/2026

Authors: Paulo C. Bartolomeu

1 Objetivos

- Familiarização com o protocolo MQTT
- Publicação de informação em tópicos MQTT: sobre TCP e sobre TLS
- Instalação e configuração de um *broker* MQTT TLS

2 Introdução

A evolução da Internet das Coisas (IoT) motivou a procura por soluções de comunicação eficientes, confiáveis e de baixo consumo energético, especialmente em sistemas *embedded*, que operam com recursos limitados de *hardware*, conectividade e energia. Nesse cenário, o protocolo *Message Queuing Telemetry Transport* (MQTT) tornou-se uma das principais escolhas para comunicação em tempo-real entre dispositivos conectados.

O MQTT é um protocolo de mensagens leve, baseado no modelo de *publish/subscribe*, desenvolvido originalmente pela IBM para aplicações industriais com redes de baixa largura de banda e alta latência. Essas características o tornam especialmente adequado para dispositivos *embedded*, como os microcontroladores da família ESP32, que frequentemente operam em ambientes com conectividade limitada ou instável, como redes Wi-Fi domésticas ou rurais.

Uma das principais vantagens do MQTT reside no seu baixo *overhead* de comunicação. Contrariamente a protocolos mais pesados, como HTTP, o MQTT envia mensagens compactas, otimizadas para consumo mínimo de dados e energia sendo isso um fator crucial para dispositivos alimentados por bateria. Além disso, o protocolo oferece níveis configuráveis de Qualidade de Serviço (QoS), garantindo a entrega das mensagens mesmo em situações de perda temporária de conexão.

O modelo de *pub/sub* adotado pelo MQTT também proporciona um forte desacoplamento entre emissores e receptores de mensagens. Isso permite uma arquitetura mais escalável e modular, em que sensores, atuadores, servidores e aplicativos móveis podem interagir de forma assíncrona, sem dependência direta entre si. Outro benefício importante é o suporte a funcionalidades como retenção de mensagens e *last will*, que tornam o sistema mais robusto e tolerante a falhas.

2.1 Arquitetura *Publish / Subscribe* do MQTT

A Figura 1 ilustra a arquitetura *Publish / Subscribe* do MQTT. Como se pode observar, é adotada uma arquitetura onde a comunicação entre dispositivos é mediada por um elemento central chamado

broker. O *broker* é responsável por receber as mensagens publicadas por clientes e distribuí-las apenas àqueles que subscreveram o tópico correspondente, garantindo assim um fluxo de dados eficiente e desacoplado.

Por exemplo, num sistema em que dispositivos ESP32 monitorizam a temperatura, um cliente pode publicar valores no tópico `sensores/temperatura`, enquanto outros clientes subscritos a esse tópico recebem automaticamente as atualizações. Esta abordagem simplifica a comunicação, evita ligações diretas entre dispositivos e permite maior escalabilidade e flexibilidade, sendo especialmente vantajosa em sistemas embarcados e aplicações IoT.

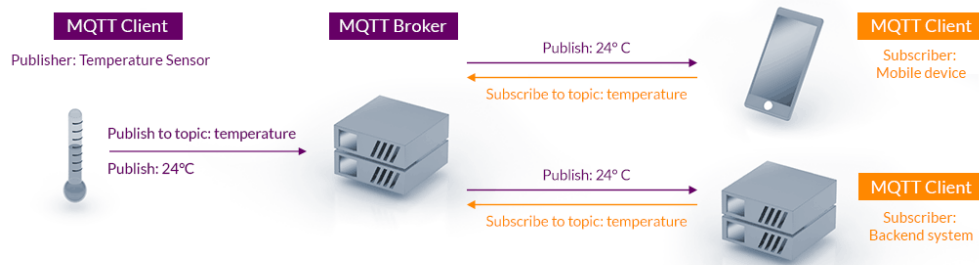


Figura 1: Arquitetura *Publish / Subscribe* do MQTT

A API [ESP MQTT](#) suportada através da *framework* ESP-IDF corresponde a uma implementação do cliente do protocolo MQTT atualmente na versão 5.0.

2.2 Qualidade de Serviço (QoS)

O protocolo MQTT define três níveis de Qualidade de Serviço (QoS) que controlam a garantia de entrega das mensagens entre clientes e o broker. Estes níveis são especialmente relevantes em sistemas *embedded*, onde a estabilidade da rede pode variar e o equilíbrio entre confiabilidade e eficiência é essencial:

- **QoS 0** – “No máximo uma vez”: A mensagem é enviada uma única vez, sem confirmação de receção. É o modo mais rápido e leve, mas não garante que a mensagem chegue ao destino.
- **QoS 1** – “Pelo menos uma vez”: A mensagem é garantidamente entregue, mas pode ser recebida mais de uma vez em caso de retransmissões.
- **QoS 2** – “Exatamente uma vez”: Garante que a mensagem é entregue apenas uma vez, sem duplicação. É o nível mais seguro, mas também o mais pesado em termos de recursos e latência.

A escolha do nível de QoS depende do tipo de aplicação: sensores com atualizações frequentes podem usar QoS 0 para eficiência, enquanto comandos críticos podem exigir QoS 1 ou QoS 2 para maior fiabilidade. A QoS é definida por mensagem enviada ou por subscrição de um tópico.

2.3 Tópico

Um tópico no protocolo MQTT corresponde a uma *string* hierárquica usada para organizar e classificar mensagens que são publicadas e subscritas. Funciona como um “canal” onde os dados são enviados (*published*) ou recebidos (*subscribed*). Os tópicos são fundamentais para o modelo *publish/subscribe*, permitindo que os dispositivos comuniquem de forma desacoplada — ou seja, sem que o emissor e o recetor se conheçam diretamente.

- Um cliente publicador envia mensagens para um tópico específico.
- Um cliente subscritor recebe mensagens ao subscrever esse tópico.

A estrutura de um tópico é semelhante a um caminho de diretórios, separado por barras (/):

- Exemplo: `casa/sala/temperatura`

Cada nível representa uma categoria ou grupo de informação, permitindo organizar os dados de forma lógica e escalável.

2.4 Modos de Utilização do MQTT no ESP32-C6

O protocolo MQTT pode ser utilizado no ESP32-C6 através de diferentes modos de transporte, adaptando-se a diversos contextos de segurança, infraestrutura de rede e requisitos de aplicação:

- **MQTT sobre TCP (`mqtt/tcp`):** Forma mais básica que utiliza a porta 1883 e não oferece qualquer camada de encriptação. Esta configuração é adequada apenas para ambientes controlados e de confiança, como redes locais (LAN) ou VPNs, onde a segurança é garantida por outros meios. É útil para testes ou protótipos, permitindo ligação a um broker, publicação e subscrição de tópicos simples, bem como gestão de reconexão em caso de falhas de Wi-Fi. No entanto, por transmitir credenciais e dados em texto claro, não deve ser utilizada em redes abertas ou na Internet.
- **MQTT sobre TLS (`mqtt/ssl`):** Modo recomendado para aplicações em produção ou com exposição à Internet e opera na porta 8883. Este modo adiciona encriptação *end-to-end* através de certificados digitais, assegurando a confidencialidade e integridade dos dados. No ESP32-C6, é possível embutir certificados de autoridade (CA) no *firmware* e validar o certificado do servidor, além de suportar, opcionalmente, autenticação mútua com certificado de cliente. Este modelo é amplamente adotado em soluções baseadas na nuvem, como AWS IoT ou Azure, embora exija cuidados adicionais com o ciclo de vida dos certificados, o consumo de memória e políticas de reconexão e *backoff*.
- **TLS com chaves pré-partilhadas (`mqtt/ssl_psk`):** Variação mais leve que substitui os certificados por uma chave simétrica comum entre o cliente e o *broker*. Esta abordagem reduz a complexidade criptográfica e o uso de memória, tornando-se útil em ambientes fechados, como laboratórios ou redes industriais privadas. Contudo, a gestão segura das chaves partilhadas é complexa em larga escala, e muitos *brokers* comerciais não suportam este método, tornando-o pouco viável para aplicações públicas.
- **MQTT sobre WebSocket (`mqtt/ws`):** Alternativa que encapsula o tráfego MQTT dentro de uma ligação HTTP via *WebSocket*, tipicamente na porta 80. Esta abordagem é útil quando a infraestrutura de rede bloqueia conexões TCP diretas, mas permite tráfego HTTP, como em redes corporativas com políticas restritivas. No entanto, quando usado sem TLS (`ws://`), a comunicação permanece vulnerável. Por isso, em cenários reais, recomenda-se a **versão segura com TLS (`mqtt/wss`)**, que utiliza a porta 443 e combina as vantagens do *WebSocket* com a segurança do TLS.
- **MQTT sobre WebSocket seguro com TLS (`mqtt/wss`):** Eficaz em contextos empresariais ou redes públicas, onde é comum que apenas a porta HTTPS esteja acessível. Além de garantir a segurança da comunicação, permite que o *broker* MQTT funcione atrás de balanceadores de carga (*load balancers*), proxies HTTP ou CDNs, partilhando a infraestrutura com outros serviços *web*. Essa configuração é ideal para dispositivos IoT que operam em ambientes

com *firewalls* restritivos, e é frequentemente adotada como a solução padrão em arquiteturas modernas de IoT.

2.5 Opções de *broker* MQTT

Existem diversos *brokers* MQTT disponíveis para instalação local ou em servidores, cada um com características específicas que se adaptam a diferentes cenários de utilização. Um dos mais populares é o Eclipse Mosquitto, uma solução leve, de código aberto, mantida pela Eclipse Foundation. É ideal para testes locais, aplicações com ESP32 e projetos de pequena a média escala. Suporta as versões 3.1, 3.1.1 e 5.0 do MQTT, além de funcionalidades como autenticação por utilizador e palavra-passe, encriptação TLS e persistência de mensagens. A sua instalação é simples, com pacotes disponíveis para sistemas Windows, Linux, macOS e também via Docker.

Para aplicações de maior escala e ambientes empresariais, o EMQX destaca-se como uma das opções mais completas. Este *broker* é altamente escalável, suportando milhões de conexões simultâneas, e inclui uma vasta gama de funcionalidades, como autenticação avançada (incluindo OAuth2 e JWT), controlo de acesso (ACL), suporte a MQTT v5, integração com bases de dados e protocolos externos, além de uma interface web para monitorização e gestão. Pode ser instalado localmente ou em containers Docker e Kubernetes, e está também disponível em versão *cloud*.

Outra alternativa robusta é o HiveMQ CE (Community Edition), a versão gratuita e open-source do *broker* HiveMQ. Foca-se na fiabilidade, desempenho e integração com sistemas empresariais. Suporta MQTT v3.1.1 e v5, e é extensível através de plugins em Java, permitindo adaptar-se facilmente a diferentes arquiteturas. A instalação é também multiplataforma, com suporte para sistemas operativos modernos e *deployment* via Docker.

Por fim, o RabbitMQ, amplamente conhecido como *broker* para o protocolo AMQP, pode também funcionar como *broker* MQTT através da ativação do plugin MQTT. Esta funcionalidade torna o RabbitMQ particularmente útil em ambientes híbridos, onde se pretende integrar dispositivos MQTT com sistemas já existentes baseados em AMQP. Com suporte a TLS, autenticação, dashboards administrativos e capacidade de interligar múltiplos protocolos, o RabbitMQ com suporte MQTT é ideal para quem já utilizam esta tecnologia e desejam expandir para aplicações IoT de forma integrada

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na [API de Referência ESP MQTT](#) e na [API de Referência Wi-Fi do ESP32-C6](#), juntamente com o [ESP32-C6 Technical Reference Manual](#).

3.1 MQTT simples sobre TCP

O exemplo de MQTT simples sobre TCP demonstra uma aplicação básica de comunicação MQTT usando o ESP32-C6, conectando-se a um *broker* através da porta padrão 1883, sem encriptação (ou seja, sem TLS). Este exemplo mostra como estabelecer uma ligação com o *broker*, configurar os parâmetros essenciais (por exemplo, URI, ID do cliente, tópico) e implementar as operações fundamentais do protocolo: publicar mensagens, subscrever tópicos e gerir reconexões automáticas em caso de falha de rede, como perda de Wi-Fi.

A configuração é feita via `menuconfig`, permitindo ao utilizador definir facilmente o URI do *broker* e outros parâmetros sem alterar o código. Este exemplo é especialmente útil para ambientes de teste

ou redes internas seguras, já que, por não usar encriptação, não é recomendado para uso em redes públicas.

O programa inicia o subsistema de rede, tenta ligar-se à rede definida no `menuconfig` e regista eventos para acompanhar o processo de ligação, incluindo tentativas de reconexão e falhas. Quando a ligação é bem-sucedida, o dispositivo obtém um endereço IPv4 via DHCP do AP, e essa informação é apresentada no terminal.

QUESTÃO 1: No VSCode crie um novo projeto a partir do template `protocols -> mqtt -> tcp`. Usando o comando `menuconfig` defina o SSID e a Password da rede Wi-Fi que tiver configurado no seu telemóvel. O acesso a esta configuração no projecto encontra-se na secção `Example Connection Configuration` do `menuconfig`. Adicionalmente, defina o Broker URL como `mqtt://test.mosquitto.org:1883`. Esta configuração está disponível na secção `Example Configuration` do `menuconfig`. Compile o programa e descarregue o *firmware* para o SoC.

Nesta fase deverá garantir que tem o seu *Access Point* corretamente configurado para permitir o acesso à Internet de modo a que o programa consiga alcançar o *broker* de teste (`mqtt://test.mosquitto.org:1883`). Assim que a placa arrancar, deverá observar na janela de *Monitor* um *log* de eventos semelhante ao seguinte:

```
...
I (23) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
...
I (291) cpu_start: Unicore app
...
I (405) main_task: Started on CPU0
I (405) main_task: Calling app_main()
...
I (425) example_connect: Start example_connect.
...
I (18235) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = 6a:9b:ac:b5:55:f8
I (18245) wifi:security: WPA2-PSK, phy:ax, rssi:-50, cipher(pairwise:0x3, group:0x3), pmf:0,
...
I (19425) example_connect: Got IPv6 event: Interface "example_netif_sta" address:
    fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (19535) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (19535) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
I (19535) example_common: Connected to example_netif_sta
I (19545) example_common: - IPv4 address: 172.20.10.2,
I (19545) example_common: - IPv6 address: fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type:
    ESP_IP6_ADDR_IS_LINK_LOCAL
I (19555) mqtt_example: Other event id:7
I (19565) main_task: Returned from app_main()
W (20685) wifi:<ba-addr>idx:1, ifx:0, tid:0, TAHI:0x100f855, TAL0:0xb5ac9b6a, (ssn:2, win:64, cur_ssn:2),
    CONF:0xc0000005
I (21385) mqtt_example: MQTT_EVENT_CONNECTED
I (21385) mqtt_example: sent publish successful, msg_id=36322
I (21385) mqtt_example: sent subscribe successful, msg_id=40368
I (21385) mqtt_example: sent subscribe successful, msg_id=8613
I (21395) mqtt_example: sent unsubscribe successful, msg_id=55753
I (21695) mqtt_example: MQTT_EVENT_PUBLISHED, msg_id=36322
I (21995) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=40368
I (21995) mqtt_example: sent publish successful, msg_id=0
I (22005) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=8613
I (22005) mqtt_example: sent publish successful, msg_id=0
I (22005) mqtt_example: MQTT_EVENT_UNSUBSCRIBED, msg_id=55753
I (22405) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
I (22615) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
```

QUESTÃO 2: Analise este resultado e o código fonte da aplicação, consultando a documentação da API. Indique a informação publicada por mensagem, o tópico correspondente e o nível de QoS associado a cada uma. Justifique a sequência de eventos apresentados no *log*.

De modo publicar, subscrever e visualizar mensagens em tópicos MQTT vamos precisar de uma ferramenta de suporte. Existem várias soluções, por exemplo o MQTT Explorer e o [MQTTX](#). Devido ao último suportar diferentes tipos de transporte (MQTT/TCP, MQTT/TLS, MQTT/WebSocket), possuir um ambiente gráfico apelativo e ser multi-plataforma, iremos optar pela sua utilização nesta aula.

QUESTÃO 3: Instale o cliente [MQTTX](#) e proceda à respetiva configuração de acordo com a Figura 2. **Nota Importante!** Atendendo a que poderão estar a aceder ao *broker* em simultâneo diferentes clientes, sugere-se que preencha o campo *client id* com um valor único que representa uma combinação da identificação da turma e do grupo.

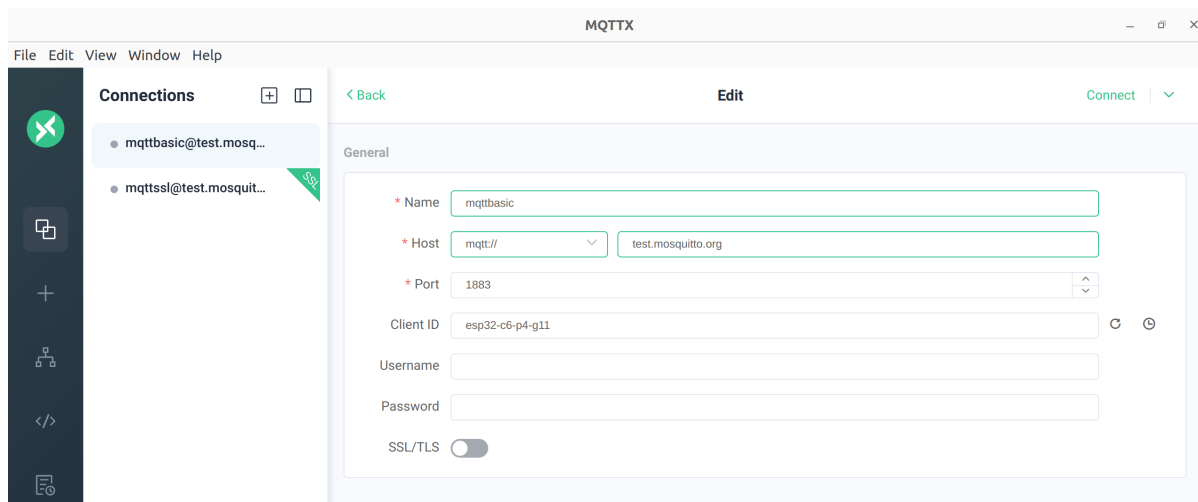


Figura 2: Configuração do cliente MQTTX

Coloque em funcionamento o cliente, suscrevendo o tópico `/topic/qos0`. Se reiniciar o programa na placa, deverá observar no cliente MQTT um resultado semelhante ao apresentado na Figura 3.

QUESTÃO 4: Cause uma interrupção momentânea no AP, por exemplo, desligando-o temporariamente 2/3 segundos e reponha-o em funcionamento. Deverá observar um comportamento consistente com a seguinte listagem:

```
I (40785) wifi:bcn_timeout,ap_probe_send_start
I (43285) wifi:ap_probe_send over, reset wifi status to disassoc
...
I (43305) example_connect: Wi-Fi disconnected 200, trying to reconnect...
E (43315) transport_base: tcp_read error, errno=Software caused connection abort
E (43325) mqtt_client: esp_mqtt_handle_transport_read_error: transport_read() error: errno=113
I (43325) mqtt_example: MQTT_EVENT_ERROR
E (43335) mqtt_example: Last error captured as transports socket errno: 0x71
I (43345) mqtt_example: Last errno string (Software caused connection abort)
E (43345) mqtt_client: mqtt_process_receive: mqtt_message_receive() returned -2
I (43355) mqtt_example: MQTT_EVENT_DISCONNECTED
I (45725) example_connect: Wi-Fi disconnected 201, trying to reconnect...
I (48125) example_connect: Wi-Fi disconnected 201, trying to reconnect...
...
I (52095) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
```

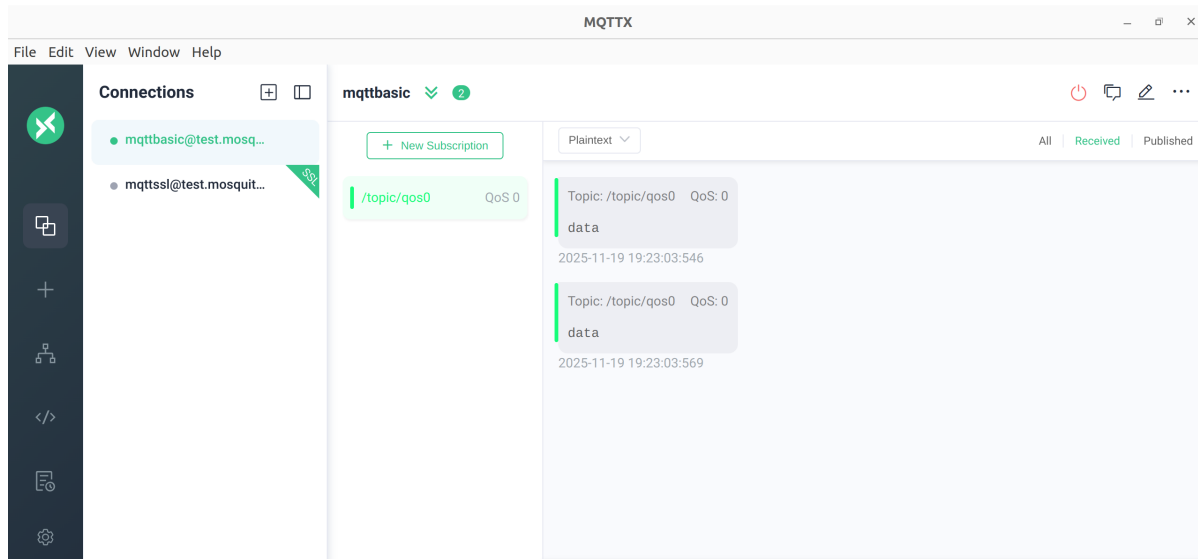


Figura 3: Configuração do cliente MQTTX

```
I (52095) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
I (52095) example_connect: - IPv4 address: 172.20.10.2,
...
I (58655) mqtt_example: MQTT_EVENT_CONNECTED
I (58655) mqtt_example: sent publish successful, msg_id=20071
I (58655) mqtt_example: sent subscribe successful, msg_id=8387
I (58665) mqtt_example: sent subscribe successful, msg_id=1962
I (58665) mqtt_example: sent unsubscribe successful, msg_id=14799
I (58755) mqtt_example: MQTT_EVENT_PUBLISHED, msg_id=20071
I (58865) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=8387
I (58865) mqtt_example: sent publish successful, msg_id=0
I (58865) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=1962
I (58865) mqtt_example: sent publish successful, msg_id=0
I (58875) mqtt_example: MQTT_EVENT_UNSUBSCRIBED, msg_id=14799
I (59065) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
I (59175) mqtt_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
```

Como se pode observar, a ligação foi ser reestabelecida e o endereço IP é novamente obtido. As mensagens MQTT são novamente publicadas.

QUESTÃO 5: Quando o cliente MQTT arranca inscreve também o tópico `/topic/qos0`. Por este motivo, recebe também as mensagens que são aí publicadas. Utilizando o cliente MQTTX publique uma mensagem neste tópico com o conteúdo `Dispositivos Conectados` e observe o terminal *Monitor* confirmando que o evento de dados correspondente (`MQTT_EVENT_DATA`) é lançado.

3.2 MQTT sobre TLS (mqtt/ssl)

O exemplo de MQTT sobre SSL incluído na plataforma ESP-IDF demonstra como estabelecer uma comunicação segura entre o ESP32-C6 e um *broker* MQTT através do protocolo *Transport Layer Security* (TLS), utilizando a porta padrão 8883. Neste exemplo, o dispositivo liga-se ao *broker* usando

a URI `mqtt://`, e o *firmware* incorpora um certificado de *Certificate Authority* (CA) para validar a identidade do servidor remoto, assegurando a confidencialidade e integridade dos dados trocados.

Tal como no exemplo MQTT simples, o código mostra como configurar o cliente, conectar-se ao *broker*, publicar mensagens, subscrever tópicos e reagir a eventos (como mensagens recebidas ou falhas de ligação). A principal diferença está na camada de segurança TLS, que exige configuração adicional, como a inclusão do certificado CA no projeto.

QUESTÃO 6: No VSCode crie um novo projeto a partir do template `protocols -> mqtt -> ssl`. Em seguida, altere o ficheiro `CMakeLists.txt` na raiz do projeto para que inclua um certificado PEM no *firmware* binário final da aplicação ESP32. Deve fazer referência ao binário gerado pelo projecto e não ao valor *default*, como se mostra em seguida:

```
target_add_binary_data(${PROJECT_NAME}.elf "main/mqtt_eclipseprojects_io.pem" TEXT)
```

Durante o processo de *build*, o conteúdo do ficheiro PEM é convertido para um array de bytes e incluído automaticamente no *firmware*. Fica depois acessível no código através de variáveis externas geradas automaticamente.

Só depois de ter realizado esta alteração é que deve prosseguir para usar o *menuconfig* para definir o SSID e a Password da rede Wi-Fi que tiver configurado no seu telemóvel. O acesso a esta configuração no projecto encontra-se na secção *Example Connection Configuration* do *menuconfig*. Adicionalmente, defina o Broker URL COMO `mqtt://test.mosquitto.org:8883`. Esta configuração está disponível na secção *Example Configuration* do *menuconfig*.

Nota Importante: Uma vez que está a usar um *broker* diferente do que é contemplado no exercício original, terá de descarregar do site `test.mosquitto.org` o ficheiro da *Certificate Authority* (CA) correspondente e substituí-lo na pasta `main` do projecto antes de poder prosseguir com a compilação.

Em seguida compile o programa e descarregue o *firmware* para o SoC. Deverá observar o seguinte *log* de eventos:

```
I (292) cpu_start: Unicore app
I (300) cpu_start: Pro cpu start user code
...
I (406) main_task: Started on CPU0
I (406) main_task: Calling app_main()
I (406) mqtt_example: [APP] Startup..
...
I (3236) wifi:connected with phobar, aid = 1, channel 6, BW20, bssid = ae:5a:9d:a1:16:45
...
I (4396) esp_netif_handlers: example_netif_sta ip: 172.20.10.2, mask: 255.255.255.240, gw: 172.20.10.1
I (4396) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 172.20.10.2
...
I (4426) example_common: - IPv4 address: 172.20.10.2,
I (4436) example_common: - IPv6 address: fe80:0000:0000:0000:5a8c:81ff:fe3b:c0bc, type:
ESP_IP6_ADDR_IS_LINK_LOCAL
I (4446) mqtt_example: [APP] Free memory: 318512 bytes
...
I (4456) main_task: Returned from app_main()
...
I (5916) mqtt_example: MQTT_EVENT_CONNECTED
I (5916) mqtt_example: sent subscribe successful, msg_id=22264
I (5916) mqtt_example: sent subscribe successful, msg_id=21810
I (5916) mqtt_example: sent unsubscribe successful, msg_id=42298
I (6016) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=22264
I (6016) mqtt_example: sent publish successful, msg_id=0
I (6116) mqtt_example: MQTT_EVENT_SUBSCRIBED, msg_id=21810
```



```
I (6116) mqttts_example: sent publish successful, msg_id=0
I (6116) mqttts_example: MQTT_EVENT_UNSUBSCRIBED, msg_id=42298
I (6216) mqttts_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
I (6326) mqttts_example: MQTT_EVENT_DATA
TOPIC=/topic/qos0
DATA=data
```

QUESTÃO 7: Explore o código e justifique a sequência de eventos MQTT apresentada na consola *Monitor*.

QUESTÃO 8: Partindo da Figura 4, configure uma sessão SSL/TLS no cliente MQTXX de acordo com os parâmetros apresentados. Note que deverá incluir o certificado CA na *box* “CA File”. Tal como para a configuração anterior do cliente MQTXX, selecione um *client ID* exclusivo, por exemplo, usando os IDs da turma e do grupo.

O *download* do certificado CA que assina o Mosquitto *test broker* em `test.mosquitto.org:8883` pode ser realizado da seguinte forma:

```
wget https://test.mosquitto.org/ssl/mosquitto.org.crt -O mosquitto_org.crt
```

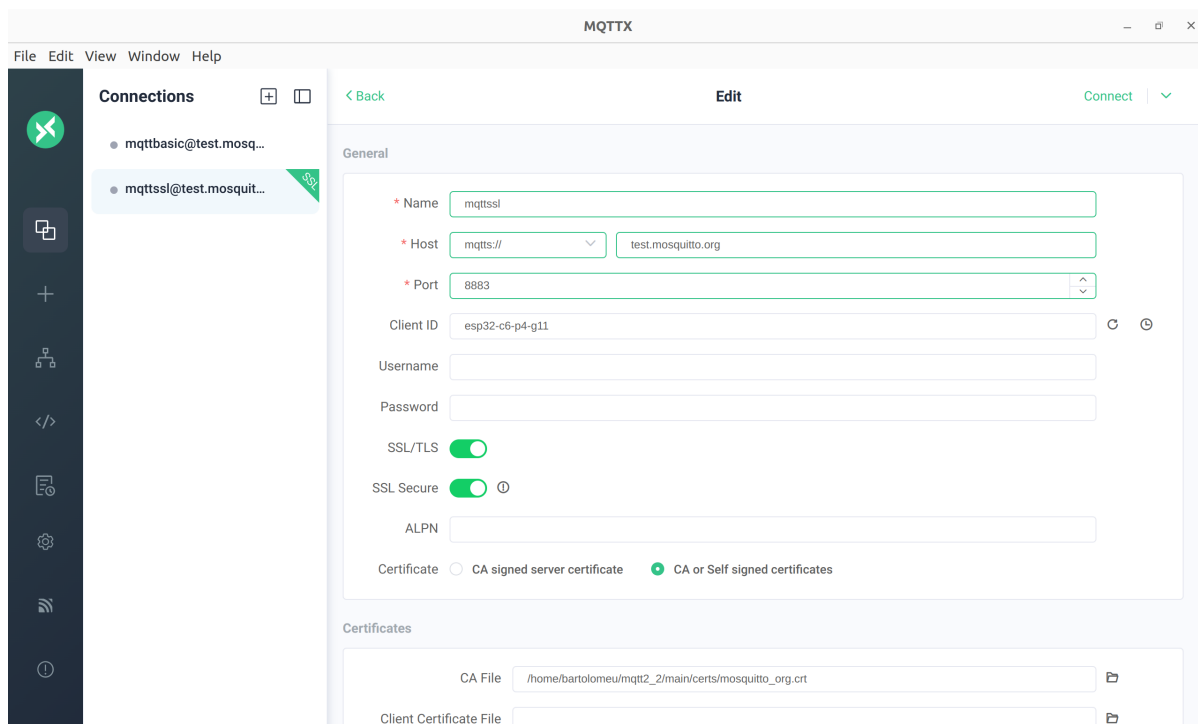


Figura 4: Configuração MQTXX para operar no modo MQTT sobre TLS (mqtt/ssl)

Se reiniciar a placa, visualizará na consola do cliente MQTXX a informação relativa à publicação de dados no tópico `/topic/qos0`, tal como ilustrado na Figura 5.

QUESTÃO 9: Tal como para o cenário de publicação MQTT sobre TCP, a aplicação do ESP32-C6 também subscreve o tópico `/topic/qos0`. Usando o cliente MQTXX publique neste tópico a informação `um dois tres` e observe o evento de dados resultante (MQTT_EVENT_DATA) no terminal Monitor do ESP32.

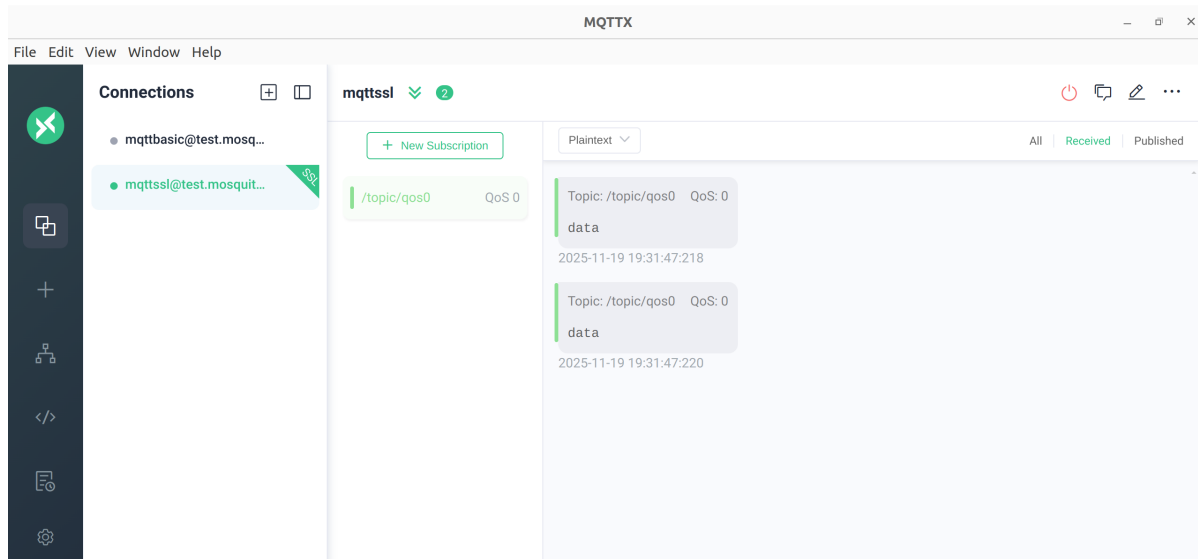


Figura 5: Monitorização do tópico `/topic/qos0` no modo MQTT sobre TLS (mqtt/ssl)

3.3 *Broker* Mosquitto

Nesta parte do guião deverá proceder à instalação do *broker* Mosquitto e realizar a respetiva configuração para que opere em ambos os modos MQTT sobre TCP e MQTT sobre TLS, tornando-o compatível com os exemplos explorados anteriormente.

NOTA IMPORTANTE!: O *broker* Mosquitto deve ser instalado no portátil pessoal tendo conectividade direta com a placa ESP32-C6 através do *Access Point* local (telemóvel), i.e., devem estar na mesma rede. Dado que o *Common Name* (CN) do certificado da CA tem de ser obrigatoriamente igual ao endereço IP do *broker* (para que a autenticação TLS não falhe), recomenda-se que a ligação do portátil ao AP seja feita logo no início e que se registe o respetivo endereço IP atribuído pelo servidor DNS a correr no AP. Isso evitará bastante trabalho mais adiante.

A instalação é simples, bastando executar os seguintes comandos na linha de comando:

```
sudo apt-get update
sudo apt-get install mosquitto mosquitto-clients
```

Em seguida gere o certificado que ficará associado ao *broker*. Note que deve substituir o IP do exemplo pelo IP do portátil onde ficará a correr o *broker*.

```
cd /etc/mosquitto/certs

# Replace 192.168.1.50 with the actual IP of your PC in the LAN
sudo openssl req -new -x509 -days 365 -nodes \
  -out mosquitto.crt \
  -keyout mosquitto.key \
  -subj "/CN=192.168.1.50"

sudo chown mosquitto:mosquitto mosquitto.*
sudo chmod 600 mosquitto.key
sudo chmod 644 mosquitto.crt
```

Em seguida, edite o ficheiro `/etc/mosquitto/mosquitto.conf` para que possua o seguinte conteúdo:

```
pid_file /run/mosquitto/mosquitto.pid
user mosquitto

persistence true
persistence_location /var/lib/mosquitto/

log_dest syslog
log_dest stdout
log_type error warning notice information

include_dir /etc/mosquitto/conf.d
```

Crie o ficheiro `/etc/mosquitto/conf.d/local.conf` com o seguinte conteúdo:

```
# Plain MQTT, no auth
listener 1883
allow_anonymous true

# TLS MQTT, no auth
listener 8883
cafile /etc/mosquitto/certs/mosquitto.crt
certfile /etc/mosquitto/certs/mosquitto.crt
keyfile /etc/mosquitto/certs/mosquitto.key
allow_anonymous true
```

Em seguida arranque o *broker* Mosquitto:

```
sudo systemctl start mosquitto
```

Para testar se este está a funcionar corretamente teste os comandos seguintes. Nota: deve substituir o endereço IP pelo correspondente do seu *broker*.

```
# Plain
mosquitto_sub -h 192.168.1.50 -p 1883 -t test -v

# TLS
mosquitto_sub -h 192.168.1.50 -p 8883 \
  --cafile /etc/mosquitto/certs/mosquitto.crt \
  -t test -v
```

Após ter realizado estas operações, terá validado o correto funcionamento do *broker* Mosquitto local.

QUESTÃO 10: Repita todos os passos anteriores, mas agora em vez de usar o broker Mosquitto remoto, deve usar o Broker Mosquitto local. **Nota Importante:** No código fonte da aplicação que usa TLS, deve realizar as alterações necessárias a que seja usado o certificado CA auto gerado neste processo de instalação e não o certificado CA original.

4 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)